ASTANA IT UNIVERSITY

School of Software Engineering

Образовательная программа Software Engineering

Отчёт к первой предзащите дипломного проекта

Разработка платформы воспроизводимого тестирования микросервисов с управлением взаимодействиями и сокращением сценариев сбоя.

Рабочее название платформы: ReproService

Команда, группа SE-2416:

Tamerlan Khassenov

Aldiyar Gaisa

Askhat Rakhmetollayev

Руководитель: Arailym Orazova, Senior-Lecturer, School of Software Engineering

Первая предзащита: 24 октября 2026 года

Дата подготовки отчёта: 14 октября 2026 года

Астана, 2026

Содержание

1 Предложение проекта 3
2 Проверка потребности 5
3 Обзор литературы 6
4 Сравнение существующих систем 10
5 Выбор технологий 12
6 Требования и критерии приёмки 14
7 Архитектура и контракты 16
8 Макет и сценарии использования 22
9 План экспериментальной оценки 23
10 План команды и риски 26
11 Согласование с руководителем 28
12 Готовность и действия команды 29
13 Список источников 32

Аннотация

Команда предлагает локальную платформу для повторного воспроизведения межсервисных сбоев, сокращения набора вмешательств и проверки исправления. Отчёт фиксирует проблему, границы MVP, анализ 23 источников, сравнение пяти систем, проект архитектуры и протокол экспериментальной оценки.

На текущем этапе подготовлены проектные документы и интерактивный макет. Рабочий backend и экспериментальные результаты ещё предстоит получить. Проверяемый вклад — влияние порядка кандидатов на стоимость сокращения при сохранении исходного нарушения. Превосходство над аналогами не заявляется до измерений.

1 Предложение проекта

RU: Разработка платформы воспроизводимого тестирования микросервисов с управлением взаимодействиями и сокращением сценариев сбоя.

EN: Development of a Reproducible Testing Platform for Microservices with Interaction Management and Failure Scenario Reduction.

KZ: Өзара әрекеттесуді басқару және ақау сценарийлерін қысқарту арқылы микросервистерді қайталанатын тестілеу платформасын әзірлеу.

Рабочее имя программного продукта — ReproService. Оно не является заявлением об уникальности торговой марки.

Команда и руководство

SE-2416, Software Engineering, Astana IT University. Tamerlan Khassenov — прокси и планирование событий; Aldiyar Gaisa — окружение, проверки и сокращение сценариев; Askhat Rakhmetollayev — интерфейс и визуализация. Arailym Orazova, Senior-Lecturer, School of Software Engineering — научный руководитель.

Проблема и предполагаемые пользователи

После выполнения операции сервис может потерять ответ. Повтор запроса способен повторно списать деньги или зарезервировать товар. Порядок сообщений меняется, поэтому обычный повтор теста может не воспроизвести ошибку. Потребность в управляемом тестировании подтверждается исследованиями S01, S02 и S05. Готовность конкретных команд пользоваться нашим инструментом пока проверяется интервью.

Пользователи: backend-разработчик, инженер автоматизации тестирования и инженер платформы. Предполагаемый заказчик — руководитель разработки команды, использующей HTTP-микросервисы. География первого пилота — Казахстан; отсутствие местных аналогов не установлено.

Цель и задачи

Цель — разработать и экспериментально оценить локальную платформу, которая сохраняет и повторяет поддерживаемые сценарии межсервисных сбоев, сокращает число вмешательств и помогает проверить исправления.

1. Описать требования на основе литературы, аналогов и 3–5 интервью.

2. Определить модель событий, вмешательств, исходного нарушения и состояния стенда.

3. Реализовать HTTP-прокси, оркестрацию и повторный запуск сценария.

4. Добавить проверки API/SQL и сокращение сценария.

5. Реализовать интерфейс истории событий и сравнения версий.

6. Измерить воспроизводимость, длину сценария и стоимость сокращения относительно базового алгоритма.

Границы MVP

Входит: один локальный Docker Compose стенд с orders, payments, inventory; HTTP/JSON; тестовая PostgreSQL; задержка запроса, потеря ответа после выполнения операции,

заданный HTTP-статус; последовательные эксперименты; JSON-сценарии; журнал событий; фиксированные проверки; простой сокращатель.

Не входит: производственная эксплуатация, Kubernetes, очереди Kafka/RabbitMQ, TLS interception, полный контроль потоков/часов/случайности, универсальный поиск первопричины, поддержка любого приложения без адаптации. Очередь разрешения независимых HTTP-событий — расширение после базового replay.

Исследовательский вопрос

Уменьшает ли сокращение набора HTTP-вмешательств с приоритизацией кандидатов по наблюдавшимся связям число пробных запусков при сохранении исходного нарушения по сравнению с обычным ddmin при одинаковом бюджете?

Основная гипотеза: число запусков уменьшится без снижения доли подтверждённых повторов. Нулевая гипотеза: устойчивого улучшения не наблюдается. Отрицательный результат допустим и должен быть отражён в выводах.

Ожидаемый результат и статус

Итог диплома: работающая платформа, воспроизводимые стенды, набор известных ошибок и исправлений, журнал измерений, документация и сравнительный анализ. На 14 октября подготовлены проектные материалы и автономный интерактивный UX-макет. Прокси, Docker-оркестрация и реальные измерения ещё не реализованы. Макет использует синтетические данные.

Практическая ценность

Ожидается сокращение времени подготовки повторяемого эксперимента и проверки исправления. Значение для бизнеса проверяется интервью и пилотом, а не наличием интерфейса или происхождением продукта. Коммерческая модель после проверки спроса: сопровождение и развёртывание для команд; цена и платёжеспособный спрос пока не установлены.

Дополнительный эксперимент: сравнить отказ до выполнения операции и потерю ответа после её успешного выполнения; протокол tests/POST_COMMIT_EXPERIMENT.md. Точное название темы при этом не меняется.

2 Проверка потребности

Статус: интервью ещё не проведены. Литература подтверждает существование класса проблем, но не спрос на наш продукт. До 18 октября получить 3–5 обсуждений с backend/QA/platform-инженерами из разных команд; минимум два собеседника должны лично разбирать межсервисные сбои.

Приглашение и согласие

«Мы студенты AITU, исследуем воспроизведение ошибок HTTP-микросервисов. Можно обсудить ваш опыт в течение 20 минут? Не нужны код, персональные данные клиентов или коммерческие секреты. С вашего согласия сохраним обезличенные заметки для диплома. Запись разговора — только с отдельного согласия».

Вопросы без подталкивания

1. Вспомните последний межсервисный сбой: что произошло и как его обнаружили?
2. Как вы воспроизводили его? Какие действия не сработали?
3. Сколько времени занял разбор? Укажите приблизительность оценки.
4. Какие инструменты использовали и чего конкретно не хватило?
5. Что мешает повторить проблему на локальном стенде?
6. Какие проверки отличают исходную ошибку от любого другого падения?
7. Как вы подтверждаете, что исправление действительно работает?
8. Какие ограничения есть на установку прокси, Docker и передачу данных?
9. Кто решает, нужен ли такой инструмент, и что потребуется для пилота?
10. После обсуждения проблемы покажите макет: какие шаги непонятны? Какой результат полезен, а какой лишний?

Не спрашивать «вам понравилась наша идея?» как доказательство спроса. Разделять рассказ о реальном случае, пожелание и интерпретацию команды.

Заполнение одной записи

ID INT-01; дата; роль и стаж; тип системы; согласие; способ контакта (хранить приватно); обезличенная цитата; описанный случай; используемый инструмент; ограничение; требование FR/NFR; подтверждённый факт/интерпретация; согласие на повторную проверку. Не публиковать контакты и запись в Git.

Правило принятия решений

Сгруппировать факты по воспроизведению, стоимости разбора и ограничениям внедрения. Для каждого изменения scope сослаться на INT-ID. Если потребность подтверждена только одной командой, назвать это пилотным кейсом. Если никто не готов установить прокси, пересмотреть способ интеграции до реализации. Отсутствие спроса также является результатом.

Реестр INT-01–INT-05 будет заполнен после интервью; подтверждающие записи пока отсутствуют.

3 Обзор литературы

Дата поиска: 14 октября 2026 года. Поиск по сочетаниям microservices, fault injection, reproducibility, delta debugging, failure minimization. Отбор: прямое отношение к поиску/повтору/сокращению сбоев, авторский или издательский источник, проверяемая библиографическая запись. Исключены маркетинговые подборки, дубли и нерелевантные работы о физических атаках. Это целевой критический обзор, не систематический обзор с исчерпывающим поиском.

Основное научное ядро: S01–S09, 9 публикаций. Восемь имеют год публикации 2021–2026; S09 2016 года сохранена как основополагающая. Для S03 указан год журнального выпуска 2022, хотя DOI относится к ранней публикации 2019 года. S08 проверена как ICSE Companion 2024, ссылка ведёт на авторскую версию. D01–D05 — технические первоисточники, они не засчитываются как научные статьи. T01–T09 обосновывают архитектурные и организационные решения. Всего 23 источника.

Аналитическая таблица

Источник	Подход и результат	Ограничение для нашего проекта	Применение
S01 · 2026	Сетевой fault injection и динамическая модель поведения сокращают исследуемые комбинации отказов.	Исследование поиска комбинаций не доказывает преимущества нашего сокращателя уже найденного сценария. Ограничения идентификации запросов важны для повторов.	Основной технический аналог; сравнивать на общей области HTTP и фиксировать правила идентификации событий.
S02 · 2021	Filibuster расширяет функциональные тесты сценариями частичных отказов и сокращает пространство проверок.	Сокращение числа тестов и минимизация конкретного сбоя решают разные задачи. Нельзя объявлять автоматическое внедрение отказов новым.	Переиспользовать проверки корректности; разделить поиск сбоя и его последующее сокращение.
S03 · 2022	Delta debugging сокращает изменения окружения микросервисов; инфраструктура автоматизирует развёртывание и параллельные проверки.	Метод работает с обстоятельствами выполнения, тогда как наша экспериментальная единица — вмешательство в HTTP-событие. Параллельность требует изоляции.	Явно определить единицу сокращения; в первой версии выполнять кандидатов последовательно.
S04 · 2024	Поиск использует покрытие и I/O-обратную связь для приоритизации комбинаций отказов.	Системное покрытие требует инструментации; эти результаты нельзя напрямую переносить на прикладной HTTP-прокси.	Начать с ограниченного бюджета и простых базовых алгоритмов; покрытие оставить расширением.

Источник	Подход и результат	Ограничение для нашего проекта	Применение
S05 · 2024	Anduril сочетает статический причинный анализ и обратную связь для воспроизведения конкретного сбоя.	Обнаружение нового сбоя отличается от воспроизведения известного. Статический анализ не входит в наш MVP.	Сохранять идентификатор исходного нарушения, не считать другую ошибку успешным повтором.
S06 · 2024	Legolas выбирает точки внедрения отказов с помощью выведенных из кода абстрактных состояний.	Тонкое управление внутренними состояниями требует инструментации; HTTP-слой не обеспечивает аналогичную наблюдаемость.	Ограничить обещания контролем наблюдаемых межсервисных событий.
S07 · 2025	CAFault учитывает конфигурации, связанные с отказами, при сокращении пространства тестирования.	Одна конфигурация не представляет все режимы работы. Наш диплом не исследует полный конфигурационный поиск.	Фиксировать тайм-ауты, число повторов, версии и контрольную сумму начальных данных.
S08 · 2024	Расширение Filibuster внедряет ошибки клиентов SQL/NoSQL и показывает результаты разработчику.	Наличие визуализации и работы с БД само по себе не даёт нашему продукту новизны.	Различать внедрение отказов БД и чтение БД для проверки бизнес-инварианта.
S09 · 2016	DEMi автоматически сокращает ошибочные выполнения распределённых систем.	Основа старше пяти лет сохранена намеренно: минимизация распределённых выполнений давно исследуется.	Не заявлять глобальный минимум; оценивать качество сокращения и бюджет повторных запусков.
D01 · 2026	Прокси, трассировка, JUnit 5, HTTP/gRPC и ограничения причинной модели.	Факты об интерфейсе и возможностях проверяются по README, неизвестное помечается НП.	Критерии сравнения и контракт событий.
D02 · 2026	Генерация отказных вариантов из функциональных тестов.	Не приравнивать сокращение пространства к минимизации найденного сбоя.	Сравнение аналогов.
D03 · 2026	Детерминированный гипервизор, fault injection и анализ причин.	Документальное сравнение без доступа к продукту; условия автономного размещения не подтверждены текущей страницей.	Граница между полным детерминизмом и HTTP replay.
D04 · 2026	Kubernetes-ориентированная платформа экспериментов с отказами.	Повтор конфигурации эксперимента не гарантирует идентичный порядок событий.	Сравнение развёртывания и оркестрации.

Источник	Подход и результат	Ограничение для нашего проекта	Применение
D05 · 2026	Локальная запись и повтор API-трафика и зависимостей.	Record/replay не тождествен управлению всем исполнением процесса.	Сравнение replay и сценариев проверки исправлений.

Синтез и исследовательская задача

Исследования разделяются на поиск отказных комбинаций, воспроизведение известного нарушения и сокращение ошибочного выполнения. Сокращение пространства поиска нельзя автоматически считать минимизацией найденного сценария. Сравнение должно фиксировать, какую из этих задач решает алгоритм.

Тема диплома объединяет управление HTTP-событиями, восстановление состояния и сокращение сценария. Каждый компонент уже имеет аналоги. Предмет эксперимента — влияние учёта зависимостей при сокращении на число запусков и сохранение воспроизводимости. Превосходство над конкурентами и уникальность комбинации пока не доказаны.

Планируется сравнить обычный ddmin и вариант с приоритизацией кандидатов по связям на одних и тех же наборах вмешательств. Сравнивать сценарии с различной структурой связей. Обязательная валидация одинакова для всех методов; наблюдавшиеся связи могут определять порядок кандидатов, но не доказывают их невозможность. Уточнение: ADR-005 в docs/15-reducer-decision.md.

Ограничения обзора

Глубина доступа указана для каждой записи в references.json. Для части работ изучены аннотации и библиографические записи, поэтому детали алгоритмов и численные результаты не переносятся в проект как проверенные. До реализации сокращателя команда должна подробно разобрать S03 и S09, а до сравнения с Reynard — воспроизвести официальный пример S01/D01. Числа из чужих экспериментов не являются результатами нашего проекта.

Дополнительная проверка разделов полных текстов S01/S02/S09: docs/14-source-audit.md. Уточнённый эксперимент по моменту отказа: tests/POST_COMMIT_EXPERIMENT.md.

Уточнения по полным текстам

Проверено 14 октября 2026. Ниже — дополнительные выводы по разделам полных текстов. Это работа ассистента над источниками, не свидетельство личного прочтения студентами.

S01 — Reynard, ICSE 2026

Авторский PDF, §3.1–3.5, §4, §5.4. В описанной модели прокси подменяет ответ ошибочным и не отправляет исходный запрос целевому сервису. Идентификатор использует назначение, интерфейс, payload, предшественников и счётчик; опирается на предположения о поведении сервиса. Отбрасывание невозможных комбинаций и обработка зависимостей уже присутствуют. Retry reduction имеет ограничение корректности и по умолчанию отключена.

Следствие для проекта: различать ошибку до выполнения операции и потерю ответа после её успешного выполнения. Второй случай может оставить побочный эффект и вызвать повтор. Это конкретное отличие планируемой модели от модели §3.1, но не доказательство отсутствия такой функции во всех версиях инструмента. Практическая проверка актуальной реализации ещё нужна.

S02 — Filibuster, SoCC 2021

Полный PDF, §3.1.2–3.1.4, §4, Algorithm 1. Повтор отдельного найденного контрпримера прямо описан в статье. Dynamic reduction сокращает пространство тестов, а не обязательно число вмешательств в конкретном контрпримере. Предпосылки включают детерминированные ответы и отсутствие определённых зависимостей от предыдущих ошибок.

Следствие: убрать повтор сбоя из списка уникальных возможностей. Проверять не только доступность endpoint, но и устойчивость бизнес-инварианта. Не переносить гарантии сокращения без их предпосылок.

S09 — DEMi, NSDI 2016

Полный PDF, §2.3, §3.2.1, §4.1–4.2. Метод связывает сокращение внешних событий с поиском расписаний внутренних событий. Сопоставление сообщений учитывает изменяющиеся поля, а управление исполнением требует инструментации. Статья различает 1-minimal и глобальный минимум и прямо ограничивает применимость.

Следствие: порядок событий и причинная информация не являются новой идеей. Из одного наблюдавшегося порядка нельзя выводить обязательную зависимость. При недостаточном контроле исполнения публиковать долю воспроизведения и причины неуспешных повторов.

Корректировка исследовательского вклада

Защищаемый инженерный результат: локальный инструмент для заданного класса HTTP-сбоев и воспроизводимый протокол его оценки. Исследовательская часть: сравнение способов сокращения и отдельный эксперимент с ошибкой до операции против потери ответа после операции. До реализации и измерений преимущества остаются гипотезами. Утверждение «такого нигде нет» не подтверждено.

S03 пока остаётся на уровне доступной аннотации и библиографической записи; не считать детали её алгоритма полностью проверенными. Остальная глубина доступа записана в references.json.

4 Сравнение существующих систем

Срез документации: 14 октября 2026 года. Все пять систем сравниваются по одинаковым десяти критериям. НП означает, что возможность не подтверждена изученными материалами; это не утверждение об её отсутствии. Фактическое развёртывание аналогов не выполнено. Наша колонка описывает целевой MVP, а не готовые возможности. Источники: S01/S02/S08 и D01–D05.

Критерий	**Reynard**	**Filibuster**	**Antithesis**
1 Основная задача	Поиск комбинаций частичных отказов	Отказные варианты функциональных тестов	Исследование поведения всей системы
2 Уровень управления	Сетевые прокси	Вызовы сервисов/клиентов	Детерминированный гипервизор
3 Развёртывание	Локальный/тестовый стенд	Среда разработки и CI	Среда Antithesis; условия on-prem НП
4 Интеграция	Прокси, trace context, JUnit	Инструментация и тесты	Передача окружения и свойств
5 Внедрение отказов	До отправки upstream в модели §3.1 S01	Ошибки удалённых вызовов	Fault injection
6 Полный детерминизм	Нет общей гарантии	НП	Заявлен разработчиком
7 Повтор сценария	Повторяемые тесты	Replay найденного контрпримера, §3.1.4 S02	Детерминированный replay
8 Сокращение найденного сбоя	Отдельный reducer НП	Снижение пространства поиска; reducer НП	Анализ причин; наш формат reducer НП
9 Проверка корректности	Утверждения теста	Утверждения теста	Инварианты/свойства
10 Представление результата	Тесты и трассы, опционально Jaeger	Результаты тестов, IDE-визуализация	Трассы и анализ причин

Критерий	**Chaos Mesh**	**Keploy**	**Наш MVP**
1 Основная задача	Chaos-эксперименты	Запись и повтор API	Повтор и сокращение конкретного сбоя
2 Уровень управления	Инфраструктура/сеть	Трафик и зависимости	Выбранные HTTP-события
3 Развёртывание	Kubernetes	Локально/CI	Docker Compose
4 Интеграция	Установка в кластер	CLI и захват трафика	Прокси, идентификаторы, seed данных
5 Внедрение отказов	Сетевые и инфраструктурные сбои	Не основной сценарий обзора	До вызова и потеря ответа после успешного upstream
6 Полный детерминизм	НП	НП	Не заявляется
7 Повтор сценария	Повтор конфигурации	Record/replay API	Повтор вмешательств и начального состояния
8 Сокращение найденного сбоя	Автоматический reducer НП	Автоматический reducer НП	Бюджетное сокращение вмешательств

Критерий	Chaos Mesh	Keploy	Наш MVP
9 Проверка корректности	Проверки эксперимента	Сравнение ответов, проверки	API/SQL-инварианты
10 Представление результата	Dashboard	Отчёты тестов	Таймлайн, сценарий, сравнение версий

Вывод для проекта

Ближайшие аналоги — Reynard и Filibuster. Локальный запуск, прокси, Docker и тесты сами по себе преимуществом не являются. Antithesis предоставляет существенно более глубокий контроль исполнения. Нельзя на основании этой таблицы утверждать, что ни у одного продукта нет сокращателя: пометка НП требует дальнейшей проверки.

Проверяемая гипотеза полезности продукта — единый локальный процесс сохранения, сокращения и проверки одного бизнес-сбоя. Измеримое алгоритмическое сравнение проводится с обычным ddmin на том же стенде; с Reynard — только на совпадающих поддерживаемых сценариях. До пилота запрещено утверждать, что решение быстрее, дешевле или точнее всех аналогов.

Конкретное целевое отличие относительно модели §3.1 статьи S01 — вмешательство после успешного выполнения upstream. Оно проверяется отдельным PRE/POST-экспериментом; актуальная реализация аналога практически не проверена.

5 Выбор технологий

Решения предварительные, их подтверждает руководитель и короткий технический пилот. Оценка качественная: производительность не измерялась. Источники T01–T08 и D04; выбор отражает требования проекта.

Решение	Альтернатива 1	Альтернатива 2	Выбор и основание	Условие пересмотра
Прокси и контроллер	Go: HTTP-библиотека и конкурентные операции; команде нужен пилот	Python asyncio: быстрое прототипирование, знакомый стек; иной подход к конкурентности	Go для целевого MVP. В одном процессе удобно связать HTTP-перехват и планировщик; это инженерное решение, не доказательство большей скорости	Если технический пилот не даёт нужного контроля ответов, пересмотреть до интеграции
Метаданные	PostgreSQL: сервер, транзакции, несколько клиентов	SQLite: встроенная БД, простая локальная установка	PostgreSQL для метаданных; отдельная тестовая БД для SUT. Нужна изоляция данных экспериментов от состояния приложения	Если издержки установки мешают интервью-пилоту, рассмотреть SQLite для метаданных
Окружение	Docker Compose: один локальный многоконтейнерный стенд	Kubernetes: кластерная оркестрация, больше настройки	Docker Compose соответствует FR-01 и одному стенду. Kubernetes выходит за MVP	Только после работающего replay и реальной потребности пользователей
Интерфейс	React + TypeScript: компоненты и статическая типизация	Vue + TypeScript: альтернативная компонентная модель, опыт Тамерлана с Vue	React + TypeScript условно выбран для распределения работы; решение подтвердить с Askhat/Aldiyar	Если команда быстрее поддерживает Vue, смена до разработки интерфейса не меняет исследовательскую задачу

ADR 001 Управление на уровне HTTP

Контекст: требуется показать потерю ответа после успешной операции, сохранив наблюдаемость. Решение: прокси завершает запрос к upstream, записывает событие, затем применяет fault action к downstream. Последствие: тестируются прикладные HTTP-отказы; потоки ОС и внутренние таймеры не контролируются.

ADR 002 Идентичность сценария

Сценарий хранит версию формата, digest контейнеров, seed данных, параметры тайм-аутов/повторов, селекторы событий, инвариант и бюджет. Ключ события строится из логической операции, сервиса, метода, пути и номера попытки. Случайный trace ID помогает связать трассу, но не является стабильным ключом replay. Если селектор не сопоставлен, результат INAPPLICABLE.

ADR 003 Сокращение

Единица удаления — вмешательство, а не произвольный сетевой запрос. Кандидаты не меняют полезную нагрузку основного пользовательского действия. Перед каждой проверкой восстанавливаются данные. Отсутствие исходного нарушения, другая ошибка, тайм-аут и ошибка стенда различаются. Обычный ddmin и последовательное удаление являются baseline; приоритизация кандидатов по наблюдавшимся связям — экспериментальный вариант. Обязательная валидация одинакова. Уточнение: ADR-005.

ADR 004 Хранение и доступ

Управляющий API слушает 127.0.0.1. Прокси и тестовые сервисы находятся в изолированной Compose-сети. Заголовки авторизации и cookie исключаются из журналов. Подключение произвольного Docker socket из веб-интерфейса запрещено; допускаются только заранее описанные стенды. Для первой версии доступны доверенные локальные пользователи, многопользовательский доступ не реализуется.

6 Требования и критерии приёмки

Версия 0.1, проектные требования, ожидают проверки интервью и руководителем. Must — обязательный MVP; Should — после Must; Could — расширение. Численные пороги — цели, не результаты. Для каждой строки тест TC-<ID> определён её критерием; протокол сохраняет условия и доказательство.

ID	Приор.	Требование	Критерий приёмки	Владелец
FR01	Must	Импортировать сценарий	Валидный JSON принят; неизвестное действие и отсутствующий selector отклонены до запуска.	Tamerlan
FR02	Must	Сбросить стенд	Перед каждым кандидатом восстановлены seed и изолированная БД; при ошибке reset запуск не начинается.	Aldiyar
FR03	Must	Сопоставить HTTP-событие	operation/service/method/path/attempt дают одно совпадение; ноль или несколько → INAPPLICABLE.	Tamerlan
FR04	Must	Внедрить задержку и HTTP-ошибку	Задержка и код соответствуют сценарию; ошибка синтезируется до отправки upstream; действие отражено в журнале.	Tamerlan
FR05	Must	Потерять успешный ответ	Upstream завершает операцию, затем downstream-соединение закрывается без передачи ответа; журнал хранит оба события.	Tamerlan
FR06	Must	Проверить инвариант	API/SQL-проверка возвращает ID нарушения, ожидаемое и фактическое значение; инфраструктурный сбой не считается нарушением.	Aldiyar
FR07	Must	Повторить сохранённый сценарий	Сохранены digest, seed, timeouts и selector; несовпадение окружения блокирует запуск до явного создания новой версии.	Tamerlan
FR08	Must	Сократить вмешательства	ddmin, последовательное удаление и вариант с приоритизацией получают одинаковые сценарий, валидацию и бюджет; результат сохраняет исходный invariant ID.	Aldiyar
FR09	Must	Остановить запуск	Cancel прекращает новые кандидаты, освобождает ресурсы и сохраняет CANCELLED, не PASS.	Aldiyar
FR10	Must	Показать историю и таймлайн	Фильтры по сервису и типу события, выбранное событие раскрывает selector и результат вмешательства.	Askhat
FR11	Must	Сравнить версии	Для исходной и исправленной версии показаны digest, одинаковый seed/сценарий и отдельные результаты; нет вывода «исправлено» без повторов.	Askhat
FR12	Must	Экспортировать результат	JSON содержит schema_version, конфигурацию, outcome и ссылки на события; секретные заголовки отсутствуют.	Askhat
FR13	Should	Управлять порядком запросов	Зависимости образуют DAG; цикл отклоняется; release разрешён только после выполнения предшественника.	Tamerlan
FR14	Could	Запускать из CI	CLI возвращает различные exit codes для PASS, нарушения и инфраструктурной ошибки.	Aldiyar
NFR01	Must	Изоляция	API слушает localhost; тестовая сеть отделена; произвольный compose/SQL из интерфейса не исполняется.	Tamerlan

ID	Приор.	Требование	Критерий приёмки	Владелец
NFR02	Must	Конфиденциальность	Authorization/Cookie/Set-Cookie заменены маской; контрольный секрет отсутствует в JSON и логах.	Tamerlan
NFR03	Must	Воспроизводимость	Цель: ≥27 из 30 повторов поддерживаемого сценария дают исходный invariant ID; доля и доверительный интервал публикуются.	Aldiyar
NFR04	Should	Накладные расходы	Цель: p95 дополнительной задержки прокси без fault ≤20 ms при 100 req/s на описанном стенде; отчёт сохраняет спецификацию машины.	Tamerlan
NFR05	Must	Удобство	Цель: 4 из 5 представителей пользователей без подсказки открывают сценарий, находят нарушение и экспортируют результат за 5 минут.	Askhat
NFR06	Must	Ограничение ресурсов	Бюджет 100 запусков, включая подтверждения, либо 10 минут на сокращение; завершение по первой границе с явным BUDGET_EXHAUSTED.	Aldiyar

7 Архитектура и контракты

Редактируемые схемы: diagrams/*.drawio; читаемые версии: *.svg. Это проект архитектуры, а не описание уже работающего backend.

Границы системы

Инженер управляет платформой через браузер. Контроллер запускает только заранее разрешённый тестовый стенд Docker Compose. HTTP-прокси находится между сервисами orders, payments и inventory. Метаданные платформы отделены от PostgreSQL тестового приложения. Платформа не направляет сбои в production.

Компоненты

- Web UI: конфигурация сценария, состояние запуска, таймлайн, сравнение и экспорт.
- Controller API: валидация, сериализация запусков, жизненный цикл, отмена; localhost.
- Proxy/scheduler: сопоставление событий, внедрение вмешательств, журнал причинных связей.
- Runner/reset: проверка image digest, восстановление seed, health checks, запуск нагрузки, cleanup.
- Oracle: разрешённые API/SQL-проверки; возвращает конкретный invariant ID.
- Reducer: обычный ddmin и dependency-aware вариант; обращается к одному runner/oracle.
- Metadata store: scenario/version/run/event/candidate/assertion; append-only события и итоговый статус.

Последовательность

VALIDATING → RESETTING → READY → RUNNING → ASSERTING → COMPLETED. Из каждого рабочего состояния возможны CANCELLED и INFRA_ERROR. COMPLETED отдельно содержит PASS, ORIGINAL_FAILURE, OTHER_FAILURE, TIMEOUT или INAPPLICABLE. TIMEOUT не равен исходному нарушению, кроме заранее определённого инварианта времени с тем же ID.

Перед кандидатом удаляется только принадлежащий этому run тестовый volume; посторонние контейнеры не затрагиваются. После reset сверяются хеш seed и health checks. До запуска нельзя выбрать произвольную shell-команду или SQL: используются ID заранее проверенных фикстур и проверок.

Событие и стабильное сопоставление

Event: event_id, run_id, logical_operation, source, target, method, normalized_path, attempt, parent_event_id, monotonic_offset_ms, action_id, phase, result. traceparent/tracestate [T06] связывают трассу, но случайный trace ID не является ключом повторного сопоставления.

Selector: logical_operation + source + target + method + normalized_path + attempt. Для MVP номер попытки детерминированный и задаётся тестовым драйвером; конкурентные неоднозначные совпадения запрещены. Если событие не наступило в окне ожидания, исход INAPPLICABLE. Прокси удаляет чувствительные заголовки из журнала; тела запросов по умолчанию не сохраняются.

Сокращение

1. Подтвердить исходное нарушение на полном наборе вмешательств.

2. Разделить множество вмешательств на подмножества, проверять удаления стандартным ddmin.

3. Общие явные предпосылки сценария проверять одинаково для всех алгоритмов. В варианте dependency-aware использовать наблюдавшиеся связи для порядка кандидатов; не отбрасывать кандидата только по связи в исходной трассе. См. ADR-005.

4. Для каждого допустимого кандидата выполнить reset, сценарий и oracle. Принимать только ORIGINAL_FAILURE с тем же ID; подтверждать принятый кандидат тремя повторами (3/3).

5. Остановиться при невозможности удаления, лимите бюджета или отмене. Сохранить лучший подтверждённый кандидат и причину остановки.

Повторные подтверждения расходуют общий бюджет. При флуктуациях кандидат отклоняется и помечается unstable. Исходный полный сценарий тоже проверяется одинаково для обоих алгоритмов. Удаляются вмешательства, а не бизнес-запросы. Нельзя заявлять глобальный минимум; 1-minimal допускается только если все одиночные удаления проверены и дают отрицательный результат в принятых условиях.

Черновой API

POST /api/scenarios → validation + version ID; POST /api/runs → 202/run ID; GET /api/runs/{id}; GET /api/runs/{id}/events?cursor=; POST /api/runs/{id}/cancel; POST /api/runs/{id}/reduce; GET /api/runs/{id}/export. Операции запуска имеют idempotency key, повторный POST не создаёт второй run. Параллельный запуск при занятом стенде получает 409. JSON schema_version=1. Ошибка содержит code, message, field и correlation_id.

Что не обещаем

Нет полного детерминизма ОС, потоков или часов. Не поддерживаются очереди, Kubernetes, произвольный TLS-трафик и production. Корректность на демонстрационном стенде не означает переносимость на любую систему.

Правила использования зависимостей

Статус: предложено после проверки литературы; ожидает review команды и руководителя.

Обнаруженная проблема

В первоначальном плане слово «зависимость» было слишком широким. Если в исходном запуске событие B произошло после A, это не доказывает, что B невозможно без A. Удаление вмешательства может открыть другой путь исполнения. Безоговорочное отбрасывание такого кандидата способно скрыть более короткий воспроизводимый сбой.

Решение

Разделить два вида связей:

1. Явная предпосылка сценария: действие B определено как разрешённое только после выполнения действия A. Она задаётся контрактом сценария, проверяется валидатором и применяется одинаково во всех сравниваемых алгоритмах.

2. Наблюдавшаяся связь: получена из исходной трассы. Используется для приоритизации кандидатов, но не является достаточным основанием считать их невозможными.

Базовый вариант — ddmin с общей проверкой корректности сценария. Экспериментальный вариант — та же проверка плюс выбор порядка удаления блоков на основе наблюдавшихся связей. При одинаковом score использовать стабильный порядок ID. Такой дизайн не создаёт искусственного преимущества за счёт отсутствия обязательной валидации у baseline.

Черновая эвристика: сначала пробовать удалять блоки, которые слабо связаны с событиями проверяемого бизнес-инварианта; связь задаётся прозрачной функцией по трассе. Не называть близость в графе доказанной причинностью. Формулу score зафиксировать до основного эксперимента; результаты её настройки на отладочных сценариях не включать в итоговую оценку.

Дополнительный baseline

Простое последовательное удаление по одному вмешательству с повторным проходом после принятого удаления. Оно показывает, оправдывает ли усложнение алгоритма затраты относительно понятного инженерного решения. Все методы используют один runner, oracle, кэш и одинаковый бюджет подтверждений.

Проверки корректности

- B встречается после A, но существует без A: кандидат не отбрасывается по одной трассе.
- B требует A по явному контракту: кандидат отклонён одинаково всеми алгоритмами.
- Кандидат вызывает другой invariant ID: не принимается.
- Все графовые scores одинаковы: результат порядка стабилен.
- Бюджет кончился во время подтверждений: кандидат не считается подтверждённым.

Последствия

Требуется обновить формулировку H1: влияние порядка кандидатов на стоимость сокращения, а не «пропуск заведомо невозможного без общей валидации». Возможен нулевой или отрицательный эффект. Это допустимый результат исследования.

Контекст системы

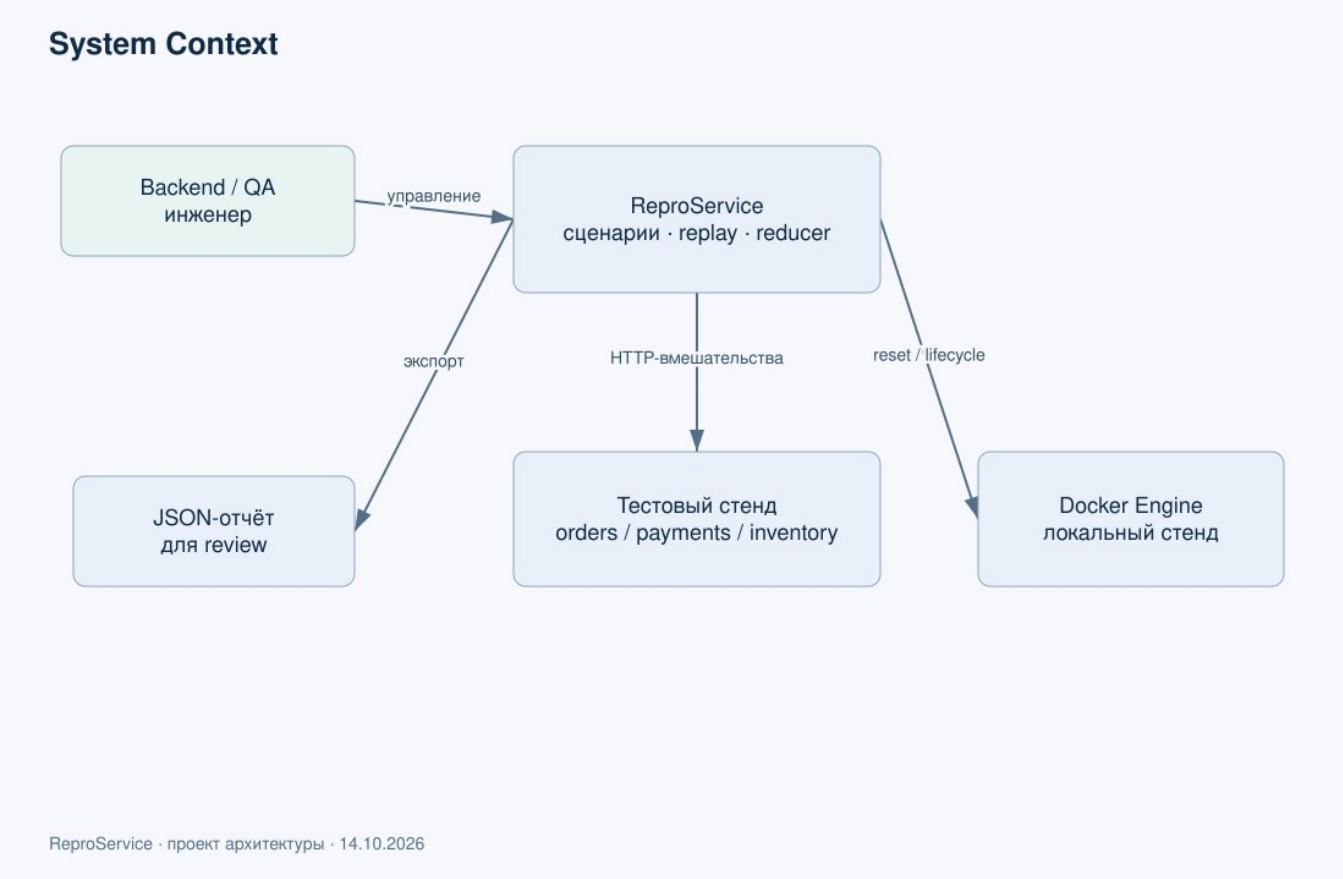


Рисунок 1 — Контекст системы

Схема отражает проектное решение. Редактируемый оригинал находится в каталоге diagrams репозитория.

Варианты использования

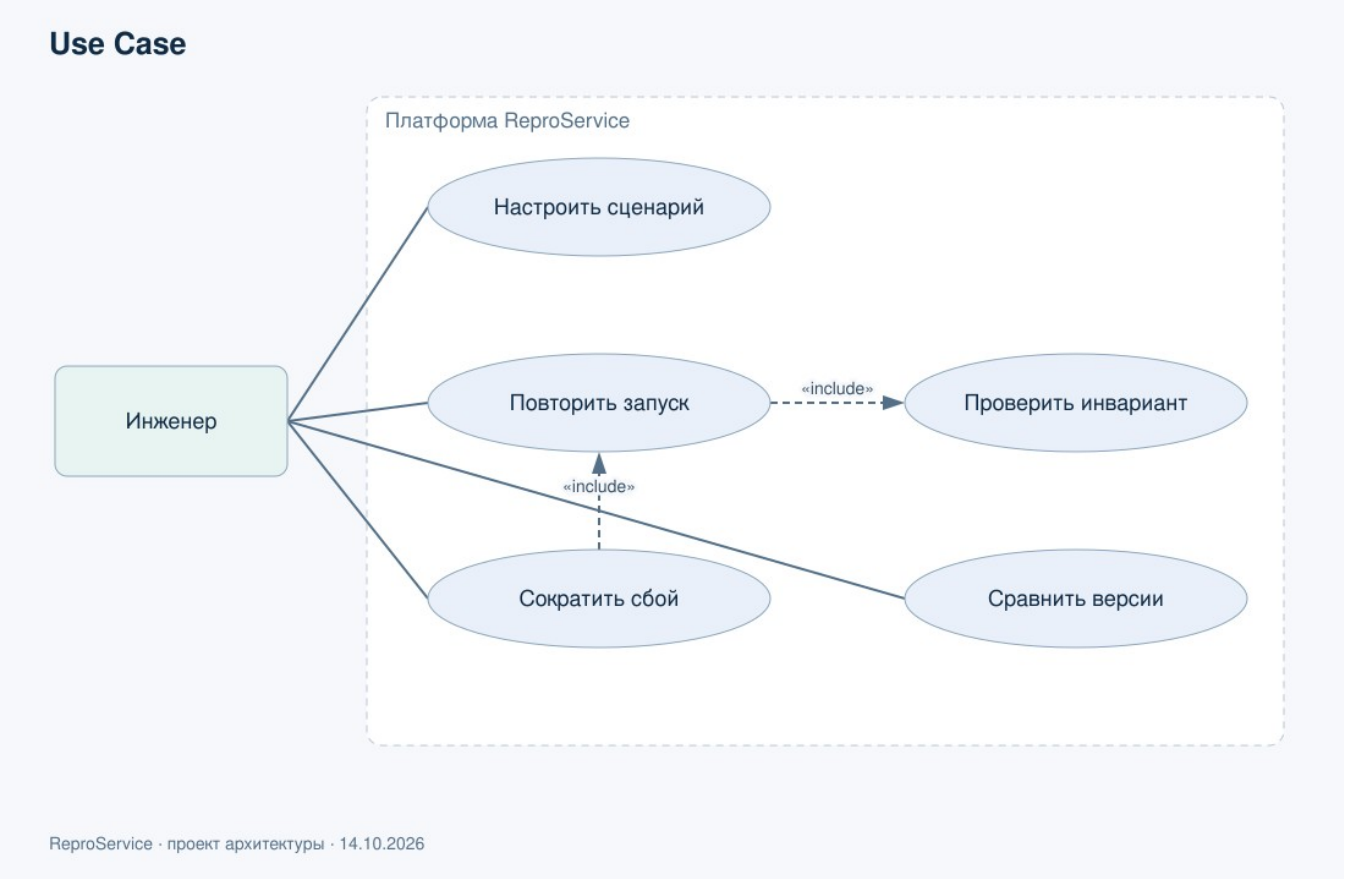


Рисунок 2 — Варианты использования

Схема отражает проектное решение. Редактируемый оригинал находится в каталоге diagrams репозитория.

Компоненты платформы

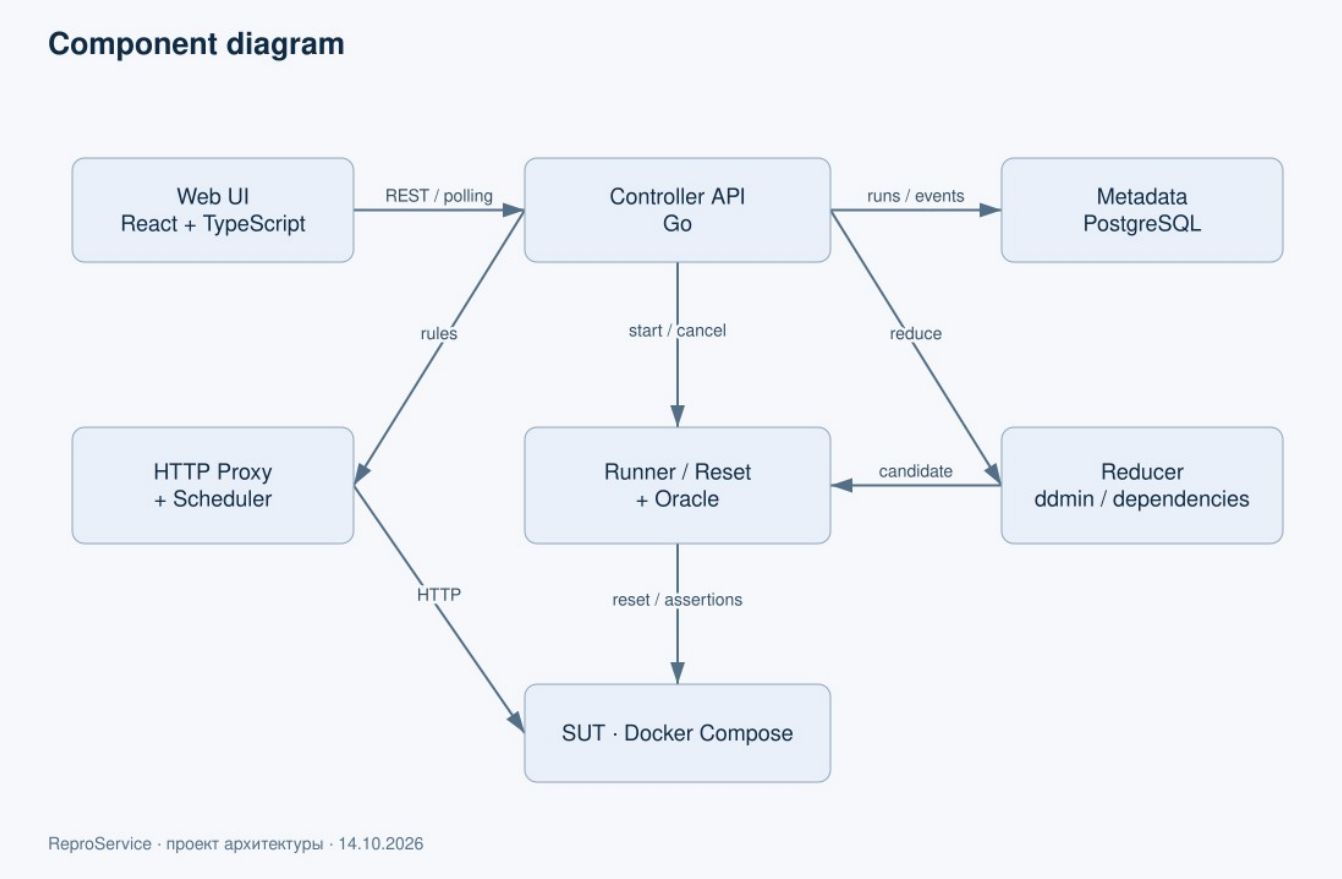


Рисунок 3 — Компоненты платформы

Схема отражает проектное решение. Редактируемый оригинал находится в каталоге diagrams репозитория.

8 Макет и сценарии использования

Открыть prototype/index.html в браузере без установки. Все числа и события синтетические; backend не подключён.

Основной путь: выбрать вмешательство → «Запустить макет» → открыть событие → увидеть нарушение → «Сократить» → сравнить с исправленной версией → экспортировать демонстрационный JSON. «Сбросить» возвращает начальное состояние. До запуска кнопки дальнейших действий недоступны.

UX-проверка: дать пользователю задачу найти причину повторной оплаты, сократить сценарий и экспортировать отчёт. Не объяснять кнопки заранее. Записать время, ошибки, непонятные формулировки и обращения за помощью. Проверить клавиатуру, контраст, перенос текста и окно 390 px. Пять реальных участников и их результаты пока отсутствуют.

Состояния будущего продукта: пустой список; валидация; запуск; отмена; исходное нарушение; другой сбой; инфраструктурная ошибка; неприменимый сценарий; бюджет исчерпан. Макет иллюстрирует основной путь, а не реализует все состояния продукта.

Шаг	Действие инженера	Показываемый результат
1	Выбрать отказ и запустить сценарий	Статус запуска и события HTTP
2	Открыть событие в таймлайне	Selector, вмешательство и invariant ID
3	Запустить сокращение	Исходный и сокращённый набор вмешательств
4	Сравнить исправленную версию	Отдельные результаты двух версий
5	Экспортировать результат	Демонстрационный JSON

9 План экспериментальной оценки

Статус: протокол до реализации. В results.csv нет измерений. Макет использует только синтетические данные.

Исследовательский вопрос

Снижает ли выбор порядка кандидатов по наблюдавшимся связям стоимость сокращения при сохранении того же нарушения по сравнению с обычным ddmin?

H1: вариант с приоритизацией по связям уменьшает число реально выполненных запусков до подтверждённого результата. H2: итоговая доля воспроизведения не ниже baseline более чем на 10 процентных пунктов. Это проверяемые гипотезы, не заявленные преимущества.

Набор сценариев

Шесть заранее описанных сценариев: потеря ответа после оплаты; повтор резерва после таймаута; ошибка inventory после успешной оплаты; задержка запроса дольше retry timeout; два независимых сбоя; зависимая цепочка из трёх вмешательств. Для каждого подготовить багованную и исправленную версию, invariant ID и фиксированный seed. Включать не только сценарии, специально выгодные новому алгоритму. Не нашедший сбой сценарий публикуется как отрицательный результат.

Условия сравнения

Одинаковые image digests, seed, CPU/RAM limits, workload, oracle, полный кандидат, порядок операций и бюджет 100 запусков / 10 минут. Перед каждым кандидатом reset. Три подтверждения принятого кандидата входят в бюджет. Считать и wall time, и число запусков. Порядок baseline/variant чередовать; отдельно выполнить 10 независимых прогонов сокращения каждого сценария каждым алгоритмом. Фиксировать фоновые процессы, ОС и версии. Сценарий — единица анализа; не считать сотни вложенных запусков независимыми наблюдениями.

Метрики

- Воспроизведение: ORIGINAL_FAILURE / 30 повторов лучшего сценария, Wilson 95% CI. INFRA_ERROR и TIMEOUT показывать отдельно, не исключать молча.
- Размер: число вмешательств до/после, доля удалений; не называть глобальным минимумом.
- Стоимость: все запуски, подтверждения, пропущенные недопустимые кандидаты, время reset и общее время.
- Бюджет: доля завершений по лимиту; для незавершённых не подставлять ноль.
- Производительность прокси: прямой запрос против прокси без fault, одинаковые 100 req/s и 60 s после прогрева; p50/p95/p99 и ошибки.

Показывать по каждому сценарию медиану и диапазон по 10 повторам; парные различия и bootstrap CI при достаточном числе сценариев. При шести сценариях выводы предварительные, обобщение ограничено. Отчёт должен включать случаи, где baseline лучше.

Функциональная проверка

TC-FR01–14 и TC-NFR01–06 соответствуют критериям docs/06-requirements.md. Негативные проверки: неоднозначный selector; цикл зависимостей; неуспешный reset; другой invariant; пропавший контейнер; отмена; исчерпание бюджета; редактированный seed; секрет в заголовках. Проверять, что ни один такой случай не ошибочно считается успешным воспроизведением.

Угрозы валидности

Один небольшой стенд, известные внесённые дефекты, нестабильная машина, субъективный выбор зависимостей, ошибки oracle. Меры: заранее фиксировать набор, версионировать протокол, сохранять сырые журналы, провести независимый review oracle и хотя бы один внешний пилот. Результаты других публикаций не смешивать с собственными измерениями.

Уточнение после разбора источников

Общая валидация сценария и кэш одинаковы для ddmin, экспериментального варианта и последовательного удаления. Третий baseline и правила обращения со связями определены ADR-005. При добавлении третьего алгоритма план 10 прогонов на сценарий применяется и к нему. Отдельный PRE/POST-эксперимент описан в POST_COMMIT_EXPERIMENT.md.

Эксперимент с потерей ответа после операции

Статус: спроектирован, не выполнен. Результаты не подставлены заранее.

Цель

Показать, какие ошибки повторных побочных эффектов обнаруживает модель потери ответа после завершения upstream, в сравнении с ошибкой, внедрённой до вызова upstream. Основание сравнения — описанная в §3.1 S01 модель, а не утверждение об ограничениях всех chaos-инструментов.

Стенд и варианты

orders вызывает payments POST /charge; при сетевой ошибке делает одну повторную попытку. Вариант BUG не обеспечивает идемпотентность. Вариант FIX сохраняет и повторно использует idempotency key, а payments гарантирует одно списание на ключ. Перед запуском seed восстанавливает пустой журнал платежей. Для данного теста inventory всегда доступен.

Режим	**Момент вмешательства**	**Ожидаемая проверка, не измеренный результат**
CONTROL	Нет отказа	Одно успешное списание
PRE	Ошибка до передачи первого запроса payments	Первая попытка не меняет ledger; проверить эффект retry
POST	payments завершает запись, затем ответ первой попытки теряется	Проверить, создаёт ли retry второе списание

Один и тот же checkout, сумма, retry policy, reset и oracle во всех режимах. Сравнить BUG и FIX во всех трёх режимах, по 30 запусков каждой ячейки. Никакие числа из UX-макета не переносить в результаты.

Oracle: число подтверждённых списаний по checkout_id, уникальность operation key и итог операции. ORIGINAL_FAILURE — число списаний >1 для исходного инварианта. Ноль списаний при неуспешном checkout — отдельный исход, а не «двойная оплата». Для FIX одного HTTP 200 недостаточно: ledger также должен пройти проверку.

Сохранить порядок upstream commit → proxy drop → retry; без подтверждения commit пометить запуск как некорректный для POST. Не моделировать POST простым HTTP 503 до upstream — это другой эксперимент.

Выводы, которые допустимы после измерений

Если POST обнаруживает дефект BUG, который PRE не обнаруживает, можно утверждать преимущество данной модели отказа на данном классе сценариев. Нельзя делать вывод о превосходстве всей платформы над Reynard или мировыми аналогами. Если обе модели находят дефект, отразить отсутствие предполагаемого отличия.

10 План команды и риски

Это план согласования, а не отчёт о выполненной студентами работе. Подготовленные с помощью ассистента материалы каждый владелец должен проверить и защитить лично.

Период	Результат	Владелец	Зависимость
14–15 октября	Проверенные scope, тема, роли, репозиторий и трекер	Tamerlan	Доступы команды
15–18 октября	3–5 интервью, чтение статей и уточнение требований	Вся команда	Согласие собеседников
18–20 октября	Архитектура, oracle, протокол эксперимента, UX-review	Aldiyar / Askhat / Tamerlan	Требования
21 октября	Обсуждение с руководителем	Вся команда	Реальная встреча
22 октября	Исправления по обратной связи, перекрёстный review	Все, не автор своего PR	Замечания
23 октября	Репетиция и проверка ссылок/экспортов	Askhat	Зафиксированные документы
24 октября	Согласованный пакет и тег pre-defense-1	Tamerlan	Review и разрешение команды

После первой предзащиты: 2 недели — стенд/прокси; 2 недели — oracle/replay; 2 недели — reducer/UI; 2 недели — эксперименты и отчёт. Это оценка трудоёмкости, даты последующих защит нужно сверить с календарём университета.

Личная ответственность и взаимное review

Tamerlan: архитектура, прокси, scheduling, интеграция. Reviewer — Aldiyar.

Aldiyar: reset, assertions, reducers, эксперимент. Reviewer — Tamerlan.

Askhat: интерфейс, таймлайн, доступность, презентация. Reviewer — Tamerlan; UX-проверка совместно с Aldiyar.

Каждый участвует в интервью и чтении источников. Нельзя приписывать подготовленные черновики личной реализованной работе.

Риски

Риск	Вероятность / ущерб	Ранний признак	Действие и владелец
Нестабильный replay	Высокая / высокий	<27/30 повторов	Сузить конкурентность и фиксировать seed; Tamerlan
Недостоверный oracle	Средняя / высокий	OTHER_FAILURE проходит как исходный	Независимый review проверки; Aldiyar
Аналог уже решает выбранную задачу	Высокая / средний	Совпадающий метод в S01/S03/S09	Уточнить экспериментальный вклад, не обещать уникальность; все

Риск	Вероятность / ущерб	Ранний признак	Действие и владелец
Нет интервью к 18 октября	Средняя / высокий	Меньше 3 согласий к 16 октября	Расширить пул инженеров, честно указать пробел; Askhat
Scope слишком велик	Высокая / высокий	Не завершены Must	Убрать FR13/FR14, оставить HTTP и последовательный стенд; Tamerlan
Зависание reset/контейнеров	Средняя / средний	Лимиты превышены	Timeout, cleanup по run ID; Aldiyar
Утечка чувствительных логов	Низкая / высокий	Тело/токен в экспорте	Allowlist полей и тест маскирования; Tamerlan

11 Согласование с руководителем

Arailym Orazova, Senior-Lecturer, School of Software Engineering, Astana IT University.

Статус: согласование и отзыв ещё не получены. Этот файл не является одобрением руководителя.

Повестка встречи, 30 минут

1. Утвердить точное название на трёх языках и границы MVP — 5 минут.

2. Обсудить новизну относительно Reynard, Filibuster, DEMi — 7 минут.

3. Проверить исследовательский вопрос, baseline и критерии исходного нарушения — 8 минут.

4. Согласовать вклад участников, сроки и условия приёмки — 5 минут.

5. Зафиксировать замечания и следующий контроль — 5 минут.

Предлагаемый контроль: еженедельная встреча 20 минут, ссылка на конкретные PR и журнал вопросов за сутки; промежуточные демонстрации после proxy, replay и reducer. Поддержка: совместный разбор нестабильных сценариев и корректировка scope при риске срыва.

Дата	Дословное замечание / согласованная формулировка	Решение команды	Владелец	Ссылка на изменение	Подтверждено руководителем
—	Ожидается реальное обсуждение	—	—	—	Нет

После встречи приложить разрешённую к публикации запись замечаний. Не подделывать подпись, дату или согласие.

12 Готовность и действия команды

Требование	Подготовленный материал	Что осталось
Предложение проекта	01-proposal.md	Личный review и согласование
Практическая актуальность	02-user-research.md	Провести 3–5 реальных интервью
≥12 источников, ≥8 научных	23 записи, 9 научных	Разделы S01/S02/S09 проверены; личное чтение и для части источников полные тексты ещё предстоят
3–5 аналогов, 6–10 критериев	5 аналогов × 10 критериев	Проверить спорные НП практическим исследованием
≥3 технологических решения	4 решения × 2 альтернативы	Утвердить ADR
Требования	FR01–14, NFR01–06	Уточнить по интервью
Context / Use Case / Component	drawio + SVG	Review связей и границ
UX-макет	prototype/index.html	Реальное пользовательское тестирование
План оценки	tests/EVALUATION.md	Реализация и измерения позднее
Git и YouTrack	Структура, backlog, инструкции	Создать удалённые проекты и настоящую историю
Отзыв руководителя	Повестка и журнал	Получить отзыв и отработать замечания
Презентация	presentation/Predefense_1.pptx	Репетиция и обновление после review

Честный статус: пакет черновиков подготовлен; формальные требования по внешним подтверждениям ещё не закрыты. До завершения review не ставить финальный тег.

Публикация и организация задач

Удалённый репозиторий и сервер YouTrack пока не подключены. PLAN-xx в backlog.json — локальные номера, не реальные issue IDs.

1. Создать приватный GitHub/GitLab repo, пригласить участников с их личными аккаунтами. Заполнить URL в README.

2. Каждый участник настраивает собственные git user.name/user.email. Не создавать коммиты от имени других участников.

3. Проверить и внести пакет первым честно датированным коммитом с указанием помощи ассистента. Дальше фиксировать реальные изменения небольшими коммитами.

4. Создать YouTrack project, добавить участников. Поля: State, Priority, Assignee, Due Date, Type. Состояния: Open → In Progress → Review → Done; Blocked отдельно. Не переводить черновики в Done без проверки.

5. Импортировать подготовленные задачи или создать вручную. scripts/import_youtrack.py сначала выполнить без --apply: он покажет тела запросов, ничего не отправляя. При реальном импорте нужны URL, ID проекта, токен в переменных окружения. Имена/даты/приоритеты будут в описании: назначить реальные custom fields вручную и сверить владельцев.

6. Ветка: feat/<real-issue-id>-short-name; PR содержит ссылку на задачу, критерий и проверку. Другой участник проводит review. main защищена от прямого push; минимум одно одобрение.

7. Перед сдачей проверить ссылки, согласование и материалы. Создать аннотированный тег pre-defense-1 только на проверенном commit. Следующие теги: pre-defense-2 и final-pre-defense. Сданные теги не переносить; исправления — новый commit и согласованный дополнительный тег.

Первый запуск на Windows

Открыть PowerShell в распакованной папке. git init -b main (если repo ещё не создан), затем git status. Настроить свою идентичность, проверить файлы, затем git add . и git commit -m "Prepare initial thesis proposal and review materials". URL remote берётся из созданного вами репозитория; не вставлять выдуманный URL. git remote add origin <REAL_URL> и git push -u origin main выполняются после создания remote.

Импорт API основан на документации JetBrains. Скрипт хранит журнал созданных IDs, но при сетевой ошибке после отправки нужно проверить сервер вручную перед повтором, чтобы не создать дубль. Токен в Git не сохранять.

Проверка списка тем

Источник: пользовательский файл Titles of Diploma Projects 2026-27 (for Students)(1).xlsx, проверка 14 октября 2026.

Проверены 50 листов, извлечена 291 пронумерованная запись темы. Выполнены нормализованное сравнение названий и поиск ключевых слов по русским и английским формулировкам. Точного совпадения с нашей темой не обнаружено. Это проверка предоставленной версии списка, а не официальное одобрение университета и не доказательство мировой уникальности.

Близкие области

Лист преподавателя / строка	Тема в списке	Различие с нашим проектом
Қожахмет Жақсылық Ерданұлы / 7	Разработка распределённой платформы обмена сообщениями и голосовой связи с низкой задержкой на основе событийно-ориентированной микросервисной архитектуры»	Микросервисы являются архитектурой коммуникационного продукта; у нас продукт — инструмент тестирования межсервисных сбоев.
Оспанов Алмас Даулетыярович / 7	Разработка системы на основе искусственного интеллекта для автоматической генерации тестовых сценариев (test cases) по документам требований с использованием локальной языковой модели	Генерация тест-кейсов из требований с LLM; у нас управление фактическими HTTP-вмешательствами и сокращение сбоя.
Өмірзақ Ислам Аманжолұлы / 6	Разработка системы автоматизированного контрактного тестирования API, генерирующей тестовые полезные нагрузки для граничных случаев на основе спецификаций OpenAPI/Swagger.	Проверка API-контракта и генерация входных данных; у нас повторные операции, fault timing и бизнес-инварианты.

Лист преподавателя / строка	Тема в списке	Различие с нашим проектом
Шутеева Гулнур Сериковна / 3	Разработка панели мониторинга результатов автоматизированного тестирования	Панель отображает тестовые результаты; наш основной вклад — управление выполнением и сокращение, а не dashboard.

Окончательное решение о допустимости и отсутствии содержательного дублирования принимает школа/руководитель. При обновлении списка проверку повторить. Контактные данные преподавателей в отчёт не включены.

Что нужно передать для завершения отчёта

Действие	Что передать	Рекомендуемый срок
Провести 3–5 интервью	Обезличенные записи: дата, роль, случай сбоя, инструменты, ограничения, согласие на использование	18 октября
Получить отзыв Arailym Orazova	Замечания по теме, scope, эксперименту и ролям; решения команды по каждому пункту	21 октября
Создать Git и YouTrack	Рабочие ссылки; доступ команды; 22 задачи с владельцами; реальные изменения участников	15 октября
Проверить отчёт всей командой	Исправления по своим разделам и подтверждение распределения работы	22 октября
Закрыть финальную проверку	Проверенные ссылки, актуальный PDF и репетиция доклада	23 октября

13 Список источников

Идентификаторы S — научные публикации, D — документация систем, T — технические источники. Дата обращения к электронным материалам: 14 октября 2026 года. Уровень изучения указан для прозрачности границ обзора.

[S01] D. Flipse, H. Simsek, J. Decouchant, B. Kulahcioglu Ozkan Automated Network-Level Fault Injection Testing of Microservice Architectures. ICSE 2026, pp. 3886–3897. 2026.

https://research.tudelft.nl/en/publications/automated-network-level-fault-injection-testing-of-microservice-a/

Изучено: Проверены разделы 3.1–3.5, 4 и 5.4 авторского полного текста; README

[S02] C. S. Meiklejohn, A. Estrada, Y. Song, H. Miller, R. Padhye Service-Level Fault Injection Testing. ACM Symposium on Cloud Computing. 2021.

https://christophermeiklejohn.com/publications/filibuster-socc-2021.pdf

Изучено: Проверены разделы 3.1.2–3.1.4 и 4 полного текста, Algorithm 1

[S03] X. Zhou, X. Peng, T. Xie, J. Sun, C. Ji, W. Li, D. Ding Delta debugging microservice systems with parallel optimization. IEEE Transactions on Services Computing 15(1), pp. 16–29. 2022.

https://ink.library.smu.edu.sg/sis_research/4844/

Изучено: Аннотация и библиографическая запись университета

[S04] W. Feng, Q. Pei, Y. Gao, D. Wang, W. Dou, J. Wei, Z. Liang, Z. Long FaultFuzz: A Coverage Guided Fault Injection Tool for Distributed Systems. ICSE Companion 2024. 2024.

https://wsdou.github.io/papers/2024-icse-faultfuzz.pdf

Изучено: Открытый текст статьи

[S05] J. Pan, H. Wu, T. Leesatapornwongsa, S. Nath, P. Huang Efficient Reproduction of Fault-Induced Failures in Distributed Systems with Feedback-Driven Fault Injection. ACM SOSP 2024. 2024.

https://www.microsoft.com/en-us/research/publication/feedback-driven-fault-injection-efficiently-reproducing-fault-induced-failures/

Изучено: Авторская аннотация и библиография

[S06] H. Wu, J. Pan, P. Huang Efficient Exposure of Partial Failure Bugs in Distributed Systems with Inferred Abstract States. USENIX NSDI 2024. 2024.

https://www.usenix.org/conference/nsdi24/presentation/wu-haoze

Изучено: Аннотация конференции

[S07] Y. Chen, F. Ma, Y. Zhou, Z. Yan, Y. Jiang CAFault: Enhance Fault Injection Technique in Practical Distributed Systems via Abundant Fault-Dependent Configurations. USENIX ATC 2025. 2025.

https://www.usenix.org/conference/atc25/presentation/chen-yuanliang

Изучено: Аннотация конференции

[S08] M. Assad, C. S. Meiklejohn, H. Miller, S. Krusche Can My Microservice Tolerate an Unreliable Database? Resilience Testing with Fault Injection and Visualization. ICSE Companion 2024, pp. 54–58. 2024.

https://arxiv.org/abs/2404.01886

Изучено: Авторская аннотация; публикация подтверждена TUM

[S09] C. Scott, A. Panda, V. Brajkovic, G. Necula, A. Krishnamurthy, S. Shenker Minimizing Faulty Executions of Distributed Systems. USENIX NSDI 2016, pp. 291–309. 2016.

https://www.usenix.org/conference/nsdi16/technical-sessions/presentation/scott

Изучено: Проверены разделы 2.3, 3.2.1 и 4.1–4.2 полного текста

[D01] Reynard contributors Reynard README. Официальный репозиторий. 2026.

https://github.com/reynard-testing/reynard

Изучено: README

[D02] Filibuster Filibuster. Официальный сайт. 2026.

https://www.filibuster.cloud/

Изучено: Официальный сайт

[D03] Antithesis Antithesis platform. Официальный сайт. 2026.

https://antithesis.com/product/

Изучено: Описание платформы

[D04] Chaos Mesh Chaos Mesh Overview. Официальная документация. 2026.

https://chaos-mesh.org/docs/

Изучено: Обзор

[D05] Keploy Keploy documentation. Официальная документация. 2026.

https://keploy.io/docs/

Изучено: Обзор

[T01] Docker Docker Compose. Официальная документация. 2026.

https://docs.docker.com/compose/

Изучено: Обзор

[T02] The Go Authors Go documentation. Официальная документация. 2026.

https://go.dev/doc/

Изучено: Обзор

[T03] Python Software Foundation asyncio. Официальная документация. 2026.

https://docs.python.org/3/library/asyncio.html

Изучено: Обзор

[T04] PostgreSQL Global Development Group What is PostgreSQL. Официальная документация. 2026.

https://www.postgresql.org/docs/current/intro-whatis.html

Изучено: Обзор

[T05] SQLite SQLite Is Serverless. Официальная документация. 2026.

https://www.sqlite.org/serverless.html

Изучено: Обзор

[T06] W3C Trace Context. W3C Recommendation. 2021.

https://www.w3.org/TR/trace-context/

Изучено: Спецификация

[T07] React Quick Start. Официальная документация. 2026.

https://react.dev/learn

Изучено: Обзор

[T08] Vue.js Introduction. Официальная документация. 2026.

https://vuejs.org/guide/introduction.html

Изучено: Обзор

[T09] JetBrains Create an Issue. YouTrack Developer Portal. 2026.

https://www.jetbrains.com/help/youtrack/devportal/api-howto-create-issue.html

Изучено: REST API guide